

გაკვეთილი 21 mod 3 (3.3)

გაკვეთილი 9 — ამოცანის იმპლემენტაცია მასივების მეშვეობით

გაკვეთილის მიზანი

ამ გაკვეთილზე მოსწავლემ დამოუკიდებლად უნდა შეძლოს:

- ამოცანის პირობის გაანალიზება;
- გადაწყვეტოს, რა მონაცემები შეინახოს მასივში;
- გამოიყენოს `for` ციკლი;
- გამოიყენოს ფუნქციები;
- გამოიყენოს `filter()`, `map()`, `find()`, `sort()`;
- საჭიროების შემთხვევაში გამოიყენოს 2D მასივი;
- მიღებული შედეგი სწორად გამოიტანოს.

ანუ აქ მთავარი კითხვა აღარ არის „რას აკეთებს ეს მეთოდი?“, არამედ:

„რომელი მეთოდი მჭირდება ამ კონკრეტული ამოცანის გადასაჭრელად?“

1. როგორ გადავჭრათ ამოცანა?

სანამ კოდის წერას დავიწყებთ, ამოცანა რამდენიმე ნაწილად უნდა დავყოთ.

ვთქვათ პირობაა:

მოცემულია მოსწავლეების ქულები. დატოვებულ მხოლოდ დადებითი ქულები, იპოვე 51-ზე მეტი ქულა და დაალაგე ისინი კლებადობით.

პირველ რიგში ვაანალიზებთ:

რა გვაქვს?

მასივი:

```
let scores = [45, 78, 20, 95, 67, 40, 100];
```

რა გვჭირდება?

51-ზე მეტი ქულები.

ესე იგი საჭიროა:

```
filter()
```

შემდეგ რა გვჭირდება?

კლებადობით დალაგება.

ესე იგი საჭიროა:

```
sort()
```

2. ამოცანის კოდად გადაქცევა

```
let scores = [45, 78, 20, 95, 67, 40, 100];

let passed = scores.filter(function(score){
  return score > 51;
});

passed.sort(function(a, b){
  return b - a;
});

console.log(passed);
```

შედეგი:

```
[100, 95, 78, 67]
```

ეს არის ამოცანის იმპლემენტაცია — პირობაში დაწერილი მოთხოვნების პროგრამულ კოდად გადაქცევა.

3. რომელი მეთოდი როდის გამოვიყენოთ?

ბავშვებს შეგიძლია ეს პატარა ცხრილი მისცე:

თუ ამოცანაში წერია...	შეიძლება დაგვჭირდეს
დაამატე ელემენტი	<code>push()</code>
წაშალე ბოლო	<code>pop()</code>
წაშალე პირველი	<code>shift()</code>
დაამატე დასაწყისში	<code>unshift()</code>
წაშალე/ჩასვი კონკრეტულ ადგილას	<code>splice()</code>
გაიარე ყველა ელემენტი	<code>for</code>
დატოვე მხოლოდ გარკვეული ელემენტები	<code>filter()</code>
შეცვალე ყველა ელემენტი	<code>map()</code>
იპოვე პირველი შესაბამისი	<code>find()</code>
დაალაგე	<code>sort()</code>
ერთი მოქმედება ბევრჯერ გამოიყენე	<code>function</code>
დააბრუნე ფუნქციის შედეგი	<code>return</code>
ცხრილის მსგავსი მონაცემებია	2D Array

ეს ძალიან კარგი შეჯამება იქნება მთელი მასივების მოდულისთვის.

4. პრაქტიკული ამოცანა — რიცხვების დამუშავება

მომხმარებელმა უნდა შეიყვანოს **10** რიცხვი.

```
let numbers = [];

for(let i = 0; i < 10; i++){

    let num = Number(prompt("შეიყვანე რიცხვი"));

    numbers.push(num);
}
```

ახლა პირობა გავართულოთ.

პროგრამამ უნდა:

- დატოვოს მხოლოდ დადებითი რიცხვები;
- გააორმაგოს ისინი;

- დააღაგოს ზრდადობით;
- იპოვოს პირველი რიცხვი, რომელიც 100-ზე მეტია.

სრული ამოხსნა

```
let numbers = [];  
  
for(let i = 0; i < 10; i++){  
  
    let num = Number(prompt("შეიყვანე რიცხვი"));  
  
    numbers.push(num);  
}  
  
let positive = numbers.filter(function(num){  
    return num > 0;  
});  
  
let doubled = positive.map(function(num){  
    return num * 2;  
});  
  
doubled.sort(function(a, b){  
    return a - b;  
});  
  
let firstBig = doubled.find(function(num){  
    return num > 100;  
});  
  
console.log("საწყისი:", numbers);  
console.log("დადებითი:", positive);  
console.log("გაორმაგებული და დალაგებული:", doubled);  
console.log("პირველი 100-ზე მეტი:", firstBig);
```

5. ფუნქციის დამატება

ახლა იგივე ამოცანაში ფუნქციაც გამოვიყენოთ.

შევქმნათ ფუნქცია, რომელიც მასივის ჯამს დაგვითვლის:

```
function calculateSum(arr){  
  
  let sum = 0;  
  
  for(let i = 0; i < arr.length; i++){  
    sum += arr[i];  
  }  
  
  return sum;  
}
```

შემდეგ:

```
let result = calculateSum(doubled);  
  
console.log("ჯამი:", result);
```

აქ უკვე ერთ ამოცანაში გვაქვს:

Array + for + filter() + map() + sort() + find() + function + return.

6. პრაქტიკული ამოცანა — პროდუქტის ფასები

გვაქვს:

```
let prices = [45, 120, 30, 250, 80, 160, 20];
```

პროგრამამ უნდა:

1. დატოვოს მხოლოდ 50 ლარზე მეტი ფასები;
2. თითოეულ ასეთ ფასს დაუმატოს 10 ლარი;
3. დაალაგოს ფასები ზრდადობით;
4. იპოვოს პირველი ფასი, რომელიც 200 ლარზე მეტია.

ამოხსნა

```
let prices = [45, 120, 30, 250, 80, 160, 20];

let expensive = prices.filter(function(price){
    return price > 50;
});

let newPrices = expensive.map(function(price){
    return price + 10;
});

newPrices.sort(function(a, b){
    return a - b;
});

let firstExpensive = newPrices.find(function(price){
    return price > 200;
});

console.log(expensive);
console.log(newPrices);
console.log(firstExpensive);
```

7. 2D მასივის გამოცდები

მე-7 გავითვლით რომ არ დაგვავიწყდეს:

```
let grades = [
    [80, 90, 70],
    [60, 100, 75],
    [95, 85, 90]
];
```

დავთვალოთ ყველა ქულის ჯამი:

```
let sum = 0;

for(let i = 0; i < grades.length; i++){

    for(let j = 0; j < grades[i].length; j++){

        sum += grades[i][j];
    }
}
```

```
}  
}  
  
console.log(sum);
```

აქ ბავშვმა უნდა გაიხსენოს, რატომ გვჭირდება **ორი for ციკლი**.

პირველი გადადის რიგებზე, მეორე კი თითოეული რიგის ელემენტებზე.



External JavaScript

index.html

```
<!DOCTYPE html>  
<html>  
<head>  
  <title>Array Practice</title>  
</head>  
  
<body>  
  
  <h1>Array Practice</h1>  
  
  <script src="script.js"></script>  
</body>  
</html>
```

script.js

აქ უკვე კონკრეტული ამოცანის კოდს დაწერენ.



დამოუკიდებელი სავარჯიშოები

ახლა მე-9 გაკვეთილის მთავარი ნაწილი პრაქტიკა უნდა იყოს.

ამოცანა 1 — ქულები

მომხმარებლისგან მიიღე 8 ქულა. პროგრამამ უნდა დატოვოს მხოლოდ 51-ზე მეტი ქულები, თითოეულს დაუმატოს 5, დააღაგოს კლებადობით და იპოვოს პირველი ქულა, რომელიც 90-ზე მეტია.

ამოცანა 2 — რიცხვები

მომხმარებლისგან მიიღე 10 რიცხვი. შექმენი ახალი მასივი, სადაც იქნება მხოლოდ ლუწი რიცხვები. შემდეგ თითოეული გაამრავლე 3-ზე და შედეგი დაალაგე ზრდადობით.

ამოცანა 3 — ფუნქცია

შექმენი:

```
function findMax(arr)
```

ფუნქციამ `for` ციკლის გამოყენებით უნდა იპოვოს და `return` -ით დააბრუნოს მასივის უდიდესი რიცხვი.

ამოცანა 4 — 2D მასივი

შექმენი:

```
let matrix = [  
  [10, 20, 30],  
  [40, 50, 60],  
  [70, 80, 90]  
];
```

იპოვე ყველა ელემენტის ჯამი, უდიდესი ელემენტი, ლუწი ელემენტების რაოდენობა და თითოეული რიგის ჯამი.

ამოცანა 5 — სრული ამოცანა

მომხმარებლისგან მიიღე 10 პროდუქტის ფასი. დატოვე მხოლოდ 50 ლარზე მეტი ფასები, თითოეულს დაუმატე 20 ლარი, დაალაგე კლებადობით, იპოვე პირველი 200 ლარზე მეტი ფასი და ფუნქციით გამოთვალე საბოლოო მასივის ყველა ფასის ჯამი.